

The method, written down

formal guardrails for LLM agents, act VI: the field manual

docs/manual.md — one compound manual teaching the three layers as one method: rules as the program, models before code, the frozen gate + agent loop — with the composition first-class.

Every transcript is real output; the worked example is a sixth committed service (Clearance) whose check.sh must PASS its gate and FAIL its preserved round-1 draft. August 2026.

No formal-methods background assumed — the four concepts you need are on slide 3.

Agents now out-write our ability to check

- LLM agents produce code faster than humans can meaningfully review it. Human attention is the bottleneck — and it samples, it does not cover.
- Tests also sample: they check the runs you thought of. The bugs that matter live in the interactions you did not think of.
- If we want agents (and background jobs, and time) to act autonomously, we need a way to say what must never break — and have a machine enforce it.

The proposal

The developer writes down intent as rules and invariants. Agents write the rest. Reasoning engines sit between them and arbitrate: every change must provably keep the rules.

Code becomes cheap to regenerate. The spec becomes the durable asset.

Setting: a regular web SaaS application — CRUD/API handlers, authorization, tenancy, workflows, background jobs. Not kernels, crypto, or avionics: every technique here is judged against ordinary product teams, CI budgets, and mainstream languages at the edges.

All the formal methods you need today

Invariant

A sentence about the system that must be true at every moment. Example: “a customer never sees another organization's cases.” You already write these — in comments, runbooks and post-mortems. Here they become machine-checkable.

Model

A small, faithful board-game version of your system: its states and legal moves. In the rule-driven layer the “model” is not beside the system — the rule base IS both the model and the running program.

Checker

A tireless adversary (Z3, Apalache, TLC). It considers every situation the rules allow — tens of thousands you would never test — hunting for one that breaks an invariant.

Counterexample

The checker's proof of failure: the exact situation that breaks the rule, with the rule that allowed it named. Not “something is wrong” but “an anonymous visitor assigned to a task can start it, via assignee_moves.”

Proofs beside the code: it works — and it has a tax

2 protocols falsified, then proven

P1: the model checker beat a careful engineer twice on an online DB migration. Version 3 survived — and was later proven inductively, for any system size, and live (tracks I, J, M).

59 anomalies per run

P3: skip the guard against real Postgres under concurrent load and the model's predicted anomaly appears 59×/run; 0 with the proven choreography.

1 round to the full fix

P4: same bug, same model, same generic prompt — a stronger frozen gate turned a partial fix into the full fix. Gate strength beats prompt steering.

3.94× faster, never wrong

P5: an agent optimized the CMS freely inside the frozen gate; two correctness-breaking “optimizations” were absorbed mechanically.

Proofs at five levels

Inductive invariants, parameterized (UPDR), liveness under fairness, noninterference via self-composition, a Dafny proof of real session code (tracks I–M).

The residual tax

Every result above polices the model↔code boundary: MBT, trace validation, extraction fidelity. Act II removed that boundary for the ruled part of the system.

Language scouting (notes 11–12): of five ways proofs can meet code, “verified engine + small analyzable DSL” — Cedar's shape — ranked first. Everything after act I builds exactly that.

The rule base is the program; the solver is the reviewer

- One artifact: rules.yaml — roles, lifecycle, allow/deny rules with their ticket sentences. The engine serving it is domain-free; refusals are 403s that NAME the rule.
- One condition grammar, two backends: the analyzer's Z3 view and the server's runtime view are the same parsed rules — agreement checked exhaustively, so spec↔code drift is closed by construction.
- The gate is frozen for agents: safety (EVERY situation) + possibility (SOME — “the safest system does nothing” fails) + feature runs with refusals expected by name + gated lifecycle entries.

```
$ analyze rulesets/cms-buggy --gate rulesets/cms # sabotage
FAIL DEAD allow rule 'editor_read_all' - silently masked
FAIL S2_separation_of_duties counterexample + granting rule
FAIL P2_review_by_non_author: IMPOSSIBLE - strict_privacy
VERDICT: FAIL (7 findings) # 4 detector kinds fire at once
```

Evidence so far (notes 13–14)

- Sabotaged CMS: 7 named findings; naive import extension: 5.
- Background jobs are just actors — a compromised importer is contained by policy, provably.
- Domain transfer: receivables cost the engine ~100 generic lines (time); Flowdeck ~60 (relations).
- Redundant rules proven dead and deleted: the solver shrinks rule bases, reversing 1980s rule rot.

Act III built a full product to measure DX. Act IV freed the UI over a kernel boundary. Act V grew relations between ruled entities. Act VI — this deck's lead — wrote the whole method down as one manual.

Flowdeck: the loop is fast and honest — the UI was the tension

1

typo = named load error; the author's dead-rule prediction FALSIFIED by the solver.

2

TWO REAL AUTHORIZATION HOLES (right) + two probes naming the wrong deny.

3

containment deny proven DEAD after the fix — deleted; S11 proves it universally.

4

PASS (0 findings). Tickets → green gate ≈ 1 hour, none of it debugging.

34,560 situations

0.18 s / round

2 real holes

round-2 draft gated forever

```
$ analyze rulesets/taskboard      # round 2, 0.18 s
FAIL S2_no_anonymous_access
  counterexample: {role: anonymous, action: start,
    assigned_to_me: true, ...}
  allowed by: assignee_moves
FAIL S5_assignee_works
  counterexample: {role: admin, action: submit,
    assigned_to_me: false, ...}
  allowed by: admin_oversees
```

Interaction bugs, not typos: each rule read fine alone. Nobody writes the test where a task is assigned to “anonymous”; the $\forall$ -check considers all 34,560 situations and names the granting rule. Full journal: [examples/taskboard/DEVLOG.md](#).

Flowdeck's UI was DERIVED from the rule base — zero app code, boards and buttons for free. Great demo, wrong product surface: a real product's UX is not a reflection of its policy vocabulary. The developer redirected.

The boundary moved into a kernel API; the UI went free

FREE — app.py (hand-written htmx, ~760 lines)

Queues, badges, toasts, forms, the thread. Agent-authored, restyled at will. Zero policy lives here — so no review of it can ever be about policy. Its product UX (a cross-state “SLA breached” queue, assign-to-me, locked-action disclosures) is nothing a reflected UI could invent.

BOUNDARY — engine/kernel.py (+ the lint that holds it)

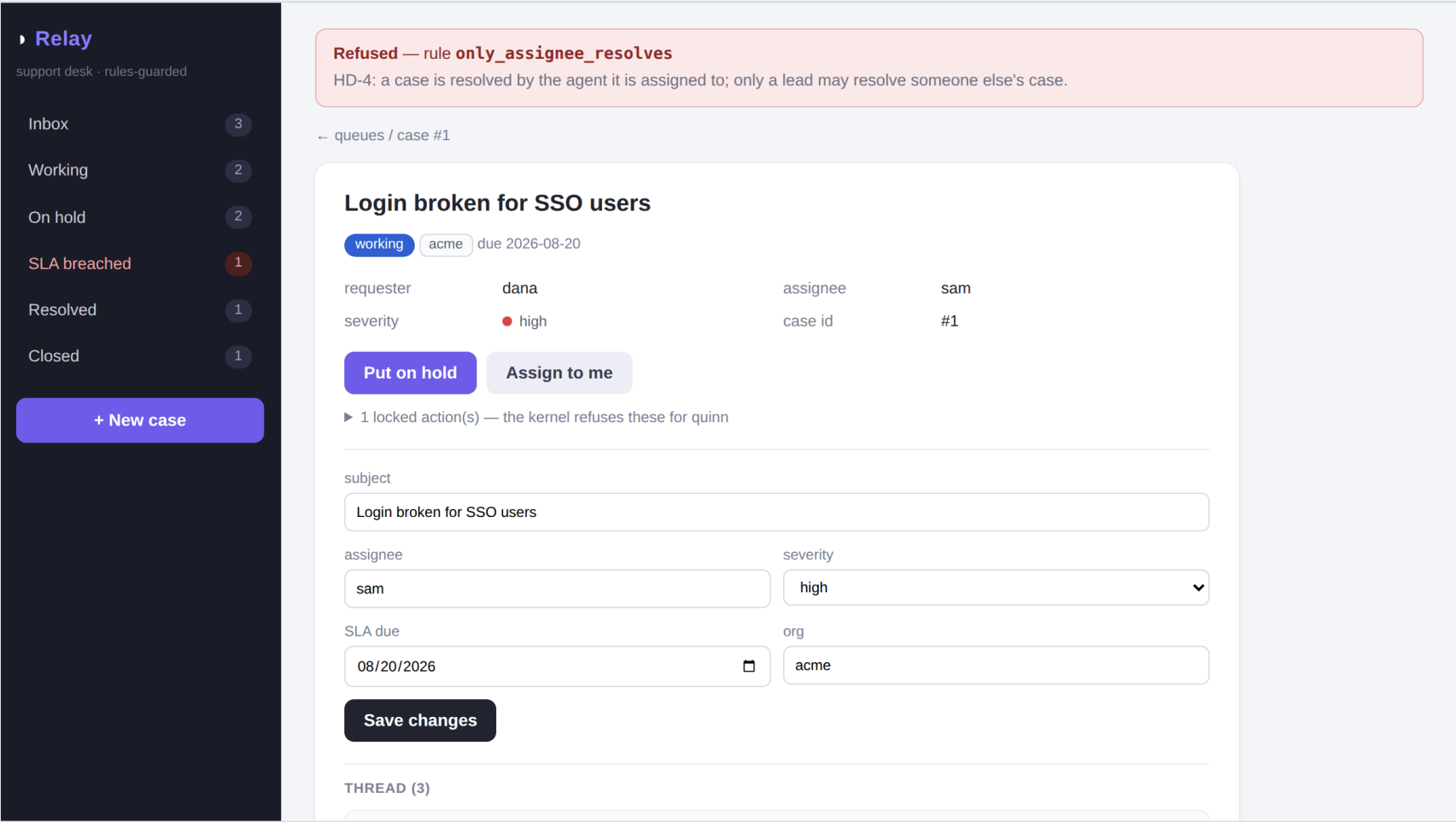
visible / get / create / act / edit / delete — every call decided by the rule base BEFORE the store is touched; refusals are typed Denied values naming the rule; decide()/affordances() are pure queries for rendering. Edits are decided twice (current row AND the row it would become — the tenant-escape edit died here). App code imports engine.kernel and NOTHING beneath it: analysis.boundary fails CI otherwise, and the preserved bypass_variant/ must keep FAILING it.

ANALYZED — rulesets/<app>/ (rules + frozen gate)

The program: allow/deny rules from tickets. Z3 reviews every change: dead rules, $\forall$ -safety, $\exists$ -possibility, lifecycle, frozen features with refusals by name. Relay round 1: PASS with pre-registered predictions held; engine domain growth for the 4th domain: 0 lines.

The tests forge every request the UI hides — the robot triaging, a non-assignee resolving, a customer re-tenanting her case — and all bounce off the kernel, 403 by rule name, nothing written. Hiding a button changes nothing; that is the whole point of the boundary.

Press the locked button anyway: the ticket sentence answers



quinn (staff, not the assignee) presses the locked Resolve: 403 — Refused: rule only_assignee_resolves, “HD-4: a case is resolved by the agent it is assigned to; only a lead may resolve someone else's case.” Ticket → rule id → solver vocabulary → typed Denied → toast: one unbroken chain of names.

Affordances, not permissions: one kernel call powers the allowed buttons AND the locked disclosures; the presentation philosophy is the UI author's choice, the decision never is.

One rule base, three entity types — the kernel joins the parent

```
children:
  - entity: comment
    context: [state, same_org] # opt-in, per atom

  - id: sealed_thread
    entity: [comment, attachment]
    effect: deny
    when: 'parent.state == "closed"
          and action != "read"'

# the kernel joins the live parent into the decision:
>>> desk.create(dana, {"body": "one more thing"},
...             entity="comment", parent_id=closed_case)
kernel.Denied: denied by sealed_thread
```

What the episode proved

- A comment's legality depends on its case's LIVE state and org — decided in the kernel, which joins the parent row. The client never computes context (it would be an enforcement point).
- Per-entity analyzer, round-1 PASS with pre-registered predictions held; internal notes do not exist for customers — THREAD (3) vs THREAD (2), even by forged id.
- The un-predicted lesson: untagged global denies fail OPEN for children (deny_inactive) — tagged, pinned by S28/S29, negative direction verified. Guardrail 11(a).
- Sharp edges recorded as falsifiers ME-1–6: no aggregates (a frame problem), cascades skip child delete rules, field-level visibility stays UI-trust.

33 rules · 3 entities

37,200 situations

0 engine domain lines

forged requests: 403 by name

“A closed case seals its thread” is one atom, checked in all 37,200 situations — not a flag the UI remembers to sync. Full story: [research/16-multi-entity-rules.md](#), [examples/helpdesk/](#).

Same case: staff see THREAD (3); the customer sees THREAD (2)

Working1

On hold1

SLA breached1

Resolved1

Closed1

+ New case

workingacmedue 2026-08-20

requesterdana

assigneesam

severityhigh

case id#1

▶ 2 locked action(s) — the kernel refuses these for dana

subject

Login broken for SSO users

assigneesam

severityhigh

SLA due08/20/2026

orgacme

Save changes

THREAD (2)

dana

Affects every SSO user since the 09:00 deploy — password logins still fine.

postbot

Fwd: user report — login loop on the EU tenant.

Write to the thread...

Post

dana's view of case #1: one comment fewer, no internal toggle. Not CSS hiding — `visible()` never returned the internal note, `get()` by forged id raises `Denied(internal_is_staff_only)`, and the forged POST with `internal=yes` comes back 403. S15 proves no allowed action ever touches an internal note outside staff.

The count in the heading is the architecture in two characters: the list IS the read rule, applied by the kernel — the UI invented the layout, the rules decided the content.

“Three best approaches” → they are layers of ONE method

The developer asked for the best approaches by developer experience, velocity and safety, each written up as a manual — then redirected: if they are layers of one method, compound them into ONE manual and care about how they compose. The three layers, by what they guard:

LAYER 1 — rules are the program

Every single-request decision: authz, tenancy, lifecycle, visibility, document discipline. The rule base is the program; Z3 checks it exhaustively — both directions — and a generic kernel enforces it under a free UI. Where you live day to day.

LAYER 2 — models before code

Everything that only goes wrong across time and interleavings: migrations under traffic, jobs, retries. Model the design (Quint), let the checker attack it, escalate the proof ladder as needed, then hold the real system to the same invariants with a conformance harness. Episodic.

LAYER 3 — the frozen gate + agent loop

How ALL code gets written: agents edit freely inside a mechanically frozen spec; counterexamples are the repair signal; objectives (speed, cost) live outside the gate. Nobody reviews agent diffs — humans review specs and counterexamples. The meta-layer.

Different failure modes need different engines — a wrong deny, a race, and a lazy agent “fix” are three different animals. The composition rules (which layer owns which concern, one vocabulary end to end, every seam mechanically held) are what make this a method rather than three tools.

docs/manual.md — 1,568 lines, self-contained, every claim runs

The shape

- Part 0 — orientation: the inversion (spec durable, code cheap), why three layers, the numbers up front.
- Part 1 — the rules layer, taught by building Clearance line by line: vocabulary as a budget, the two disciplines, the gate in three directions, the kernel, relations.
- Part 2 — models before code: the migration model, the two falsified designs, the escalation ladder, conformance.
- Part 3 — the frozen gate + loop: freeze mechanics, the controlled experiment, Grant<Op> tokens.
- Part 4 — composition: the routing table, one vocabulary, the ratchet, what stays human, adoption path.
- Part 5 — honest costs: friction ledger, falsifier index, when NOT to use this, recorded dead ends.

What makes it a manual, not a pitch

- Written for an experienced developer with NO formal-methods background: the workflow, the syntax by example, day-2 operations, the failure modes.
- Self-contained by design (developer directive): it assumes the reader reads NOTHING else — no research notes, no journals — and presents the system in its own right, not as an experiment report.
- Every transcript is real tool output, and the worked example is a real committed service with its own one-command check (next slide).
- The costs part is a priced register: the bill itemized, the sharp edges with statuses, when NOT to use it, alternatives considered.
- DX, velocity and safety get an explicit ledger per layer, and for the whole.

The manual is now the repository's front door: the root README points at it first, and every example directory points back at the part that teaches it. Adoption is staged (Part 4.7): gate one risky surface, move enforcement into the kernel, put the Layer-3 gate on one repo, reach for models at the first migration.

The worked example is executable — including its own round-1 bug

```
$ analyze rulesets/approvals-round1 \
  --gate rulesets/approvals      # the frozen gate
FAIL DEAD deny rule 'no_self_decision': it never
  refuses anything another rule would have allowed
ok   S5_no_self_decision        # passes — VACUOUSLY
FAIL P3_managers_expense_too: IMPOSSIBLE —
  no allow rule ever grants it
FAIL feat_four_eyes: step 1 (mia create):
  expected allow, got deny (rule: default_deny)
VERDICT: FAIL (3 finding(s))

$ analyze rulesets/approvals      # the fixed rule base
VERDICT: PASS (0 findings)        # 3,456 situations
```

The teaching moment, real

- Clearance: expense claims — org walls, receipts, four-eyes approval, finance-only payment, sealed records. 10 rules, 3,456 situations, gate 10 $\forall$ + 4 $\exists$ + 29 steps.
- The real round-1 draft let only employees file — so nobody could ever author-and-decide: “nobody decides their own claim” held VACUOUSLY, the deny was provably dead, the four-eyes feature could not exist.
- Safety alone said everything was fine. The possibility direction and the dead-rule check caught it — gates need both directions, demonstrated in one transcript.
- The fix was a requirement fix (managers expense too), and the broken draft is gated forever: check.sh stage 2 REQUIRES this FAIL.

[examples/approvals/](#)
[check.sh: PASS + required FAIL + HTTP replay](#)
[manual Part 1 builds it line by line](#)

The manual quotes this transcript verbatim, and the FAIL is a required CI stage — the teaching example cannot rot, because rotting would break the build.

One vocabulary, one ratchet — a real ticket across all three layers

Which layer owns which concern

- “Only X may...”, “never...”, “X sees only...” (one request decides it) → Layer 1 rule + $\forall$ -property.
- “...even while the migration runs”, “two requests at once”, “eventually” → Layer 2 model, then conformance.
- “The UI should...”, glue, engine growth → free code, agent-built under the Layer 3 gate.
- “Sum/count of...”, fuzzy matching → a projection at the boundary, or deliberately client-side — RECORDED either way.
- Incidents: reproduce the bug as a gate item, in gate vocabulary; freeze it; only then repair. The gate only ever tightens.

```
# the column rename, end to end (all committed):
falsified design #2      (L2 model trace)
-> seeded repair task      (L3)
-> partial fix exposes the
    safety-only gate gap
-> gap becomes frozen feature
    runs + witness (the ratchet)
-> full fix, 1 round,
    same generic prompt
-> inductive -> any-size ->
    liveness proofs (L2 ladder)

# every arrow is a committed artifact:
# p1, p4/episodes/, p6, p7
```

Every seam has a mechanical check: rules $\leftrightarrow$ runtime by exhaustive two-backend agreement, kernel $\leftrightarrow$ UI by the boundary lint (bypass variant must FAIL), model $\leftrightarrow$ code by the conformance harness, spec $\leftrightarrow$ agents by the frozen region diff. The one unsound seam — NL ticket $\leftrightarrow$ formal rule — is held by grounded human review plus the gate's deliberate redundancy. If a seam is held by discipline, it is already leaking.

One ghost variable (lastReadOk) carries the correctness story from the first model trace to the conformance assertion to the frozen repair gate. That continuity — not any single layer — is the method.

Six rule bases, one engine, every claim executable

6 : 1

services per engine — CMS, tickets, receivables, Flowdeck, Relay, Clearance on ~2,300 lines of domain-free Python (rule base 456, kernel 291, analyzer 351, lint 105).

111,312

situations checked exhaustively across the six rule bases — runtime evaluation and the Z3 compilation agree on every one; re-verified fresh for the manual.

1,568

lines of self-contained manual (docs/manual.md) teaching all of it — three layers + composition + priced limits, every transcript real, the worked example committed.

2-way CI

every check.sh holds both directions: gates must PASS, and the preserved buggy drafts — Clearance round-1, taskboard round-2, the bypass variant — must FAIL.

The method by the numbers:

- Layer 1: Flowdeck tickets → green gate $\approx$ 1 hour, 4 rounds $\times$ 0.18 s, 2 real authorization holes caught pre-code; Relay 33 rules over 3 entities in 721 lines of YAML, solver round $\sim$ 0.4 s.
- Layer 2: two migration designs falsified pre-code in <1 s each; proofs to every depth (inductive), every size (EPR, 0.38 s), and liveness (TLC, 623 states); the harness caught a real TOCTOU race and reproduces 59 anomalies on demand.
- Layer 3: full fix in 1 round from a stronger gate (controlled, same prompt); 3.94 $\times$ speedup with two broken attempts absorbed; capability tokens make kernel bypass a compile error (E0639/E0308, pinned).

What the analyzer asks of every rule change, in seconds, per entity: dead rules · stale assumptions · $\forall$ -safety · $\exists$ -possibility · lifecycle liveness + gated entries · frozen features with refusals by name.

Learned the hard way — the manual is their long form

11 · Relations are engine work, never client joins

Child entities import parent context (parent.state, parent projections); the kernel joins the live parent into every child decision. Sharp edges kept: untagged global denies fail OPEN (tag + pin them); cascades skip child delete rules.

1 · The spec is frozen for agents

The analyzer takes --gate from a pinned directory; agents edit rules, never the gate. Historic buggy drafts are gated forever as regressions — Clearance's round-1 joined them this week.

3 · Gate strength beats NL steering

Same bug, model, prompt: a stronger gate turned a partial fix into the full fix (P4). Cheaper models are fine when the gate is strong.

7 · Record falsified predictions

The DEVLOGs keep wrong predictions next to the runs that killed them — and the manual's Part 5 cites only that recorded evidence, never retrospect.

10 · The boundary is a kernel API, not a UI

Rules guard interaction logic behind a function-level API; the UI above is agent-authored and free; generated UIs are scaffolding. Hold the boundary mechanically (lint), never by convention.

2 · Gates need both directions

Safety-only gates accept fixes that trade away liveness (“the safest system does nothing”). Clearance's S5 passing VACUOUSLY over a dead four-eyes rule is the one-transcript demonstration.

6 · Counterexamples are the currency

Named and machine-readable: situation witnesses, unsat cores, denied_by 403s — typed Denied values any UI renders its own way.

8 · Boundaries by construction

Where the language allows: capability tokens, typestate. Where it doesn't (Python): name-mangling + a lint whose failure is named — and the honesty that this is not a proof.

Falsifiers on the table

Held so far

- RB1 (tickets land as rule diffs): held six times, and the cost curve stayed closed — imports 0, receivables ~100 (time), Flowdeck ~60 (relations), Relay 0 twice, Clearance 0 (plus one generic probe fix the new domain surfaced).
- The one-entity wall came down as engine work; both multi-entity rounds PASS with predictions pre-registered; the method survived being written down — every manual claim re-verified against the repo.

Next

- DX1 now has its instrument: hand an outside developer TICKETS.md + docs/manual.md — do they ship a ruled app in under a day without touching the engine? DX2 — the same tickets implemented conventionally by a strong LLM: does it contain the interaction holes the gates caught?
- ME-1 — aggregates, when a ticket forces them: kernel-computed aggregate projections, conservatism stated. ME-6 — cascade-decides or the load-time check. ME-5 — the fail-open lint.
- KB1 — red-team the boundary lint. KB2/ME-4 — a reporting/read seam that still applies the read rule per row. KB3 — concurrent writers between decide and write, on real Postgres (the P3 harness). KB4 — nearest-allow refusal explanations that do not leak across tenants.
- RB3 — ticket→rule-diff vs ticket→handler fidelity, measured. P8 — the same rules as Cedar policies. P9 — the invariant→Postgres-constraint compiler nobody has built.

Parked (consciously, with reasons recorded)

Alloy (Z3 covers it) · Quint→Rust codegen (no tooling) · Kani until unbounded domains arrive · Verus (proxy-blocked; Dafny stands in).

The one-sentence takeaway

You state what must never and must always happen; agents make something that does it; reasoning engines — not humans — hold the code to the statement. Six services, 111,312 exhaustively-checked situations, two falsified designs and one controlled experiment later, the method is one document.

Read it: docs/manual.md — the field manual (start at Part 0; Part 1 builds examples/approvals/)

Run it: examples/approvals/check.sh · examples/helpdesk/check.sh + python app.py · prototypes/p1..p7

The stack beneath: research/INDEX.md · notes 13–16 (rules, DX, boundary, relations) · 09–12 (proofs)